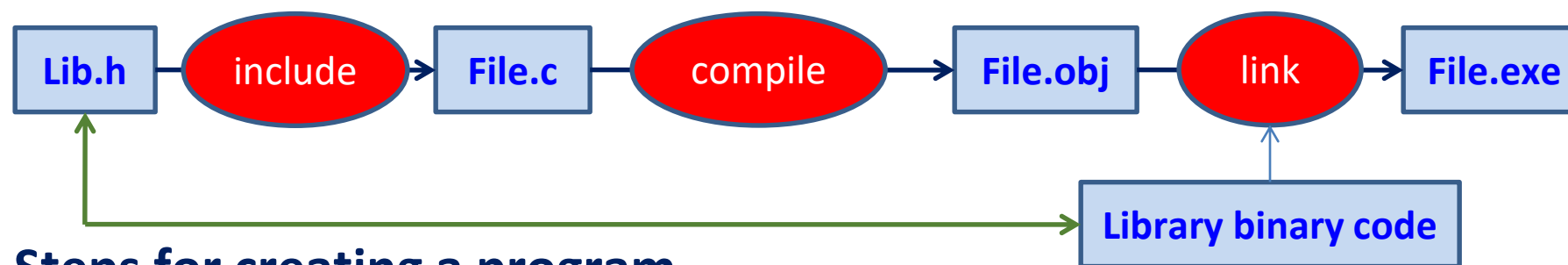


Libraries

Introduction

- ◆ Many basic tasks are very hard for programming beginners.
- ◆ The standard C libraries include functions to perform mathematical calculations, character analysis and character manipulation, ...etc.
- ◆ Library header files have the extension **.h**
- ◆ Library binary files are implemented in files with specific filenames which are named by designers of tool suppliers (you may not know their names) and they are linked to your programs when they are compiled.



Steps for creating a program

Objectives

After studying this chapter, you should be able to use the following build-in libraries:

- ◆ Standard and Math libraries: `stdlib.h`, `math.h` → Mathematical Functions
- ◆ Time library: `time.h`
- ◆ The Character library: `ctype.h`

Contents

The standard C libraries

- ◆ Standard
- ◆ Math
- ◆ Time
- ◆ Character

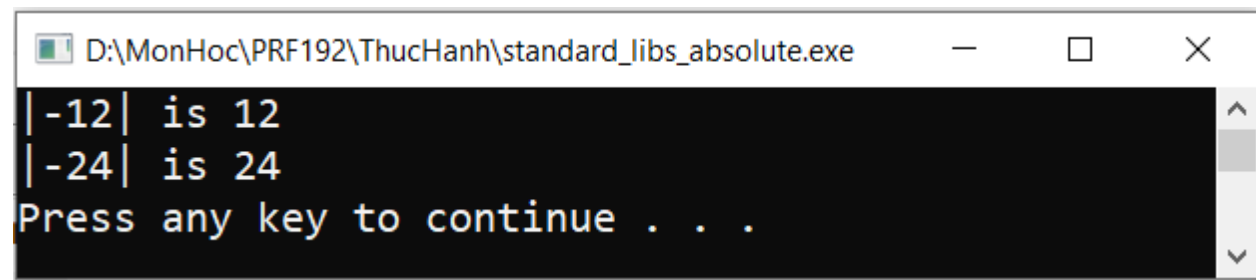
1 - Standard Library

- ◆ The header file **<stdlib.h>** contains prototypes for the functions that perform the more general mathematical calculations.
- ◆ **Integer absolute value:**

Function	Syntax	Description
abs()	<code>int abs(int);</code>	return the absolute value of the argument
labs()	<code>long labs(long);</code>	
llabs()	<code>long long llabs(long long);</code>	

Integer absolute value: Example

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  int main()
5  {
6      int x = -12;
7      long y = -24L;
8
9      printf("|%d| is %d\n", x, abs(x));
10     printf("|%ld| is %ld\n", y, labs(y));
11
12     system("pause");
13     return 0;
14 }
```



```
D:\MonHoc\PRF192\ThucHanh\standard_libs_absolute.exe
|-12| is 12
|-24| is 24
Press any key to continue . . .
```

Random Number

- ◆ **rand()**: returns a pseudo-random integer in the range **0** to **RAND_MAX**. **RAND_MAX** is implementation-dependent but **no less** than 32767.

- ◆ Syntax:

```
int rand(void);
```

- ◆ Example 1:

```
#include <stdlib.h>
#include <stdio.h>
```

```
int main()
{
    int i;

    for (i = 0; i < 10 ; i++)
    {
        printf("Random number %d is %d\n", i+1, rand());
    }

    return 0;
}
```

```
Random number 1 is 41
Random number 2 is 18467
Random number 3 is 6334
Random number 4 is 26500
Random number 5 is 19169
Random number 6 is 15724
Random number 7 is 11478
Random number 8 is 29358
Random number 9 is 26962
Random number 10 is 24464
```

Random Number: Example

- ◆ Example 2: Random a set of 10 pseudo-random integers between 6 and 100

```
#include <stdlib.h>
#include <stdio.h>

int main()
{
    int i, n, a = 6, b = 100;

    for (i = 0; i < 10 ; i++)
    {
        n = a + rand() % (b + 1 - a);
        printf("Random number %d is %d\n", i+1, n);
    }

    return 0;
}
```

```
Random number 1 is 47
Random number 2 is 43
Random number 3 is 70
Random number 4 is 96
Random number 5 is 80
Random number 6 is 55
Random number 7 is 84
Random number 8 is 9
Random number 9 is 83
Random number 10 is 55
```


Random Number: Example (cont.)

- ◆ Example 3: Random a set of 10 pseudo-random floating-point numbers between 3.0 and 100

```
#include <stdlib.h>
#include <stdio.h>

int main()
{
    int i;
    double x, a = 3.0, b = 100.0;

    for (i = 0; i < 10 ; i++)
    {
        x = a + ((double) rand() / RAND_MAX * (b - a));
        printf("Random number %d is %.21f\n", i+1, x);
    }

    return 0;
}
```

```
Random number 1 is 3.12
Random number 2 is 57.67
Random number 3 is 21.75
Random number 4 is 81.45
Random number 5 is 59.75
Random number 6 is 49.55
Random number 7 is 36.98
Random number 8 is 89.91
Random number 9 is 82.82
Random number 10 is 75.42
```

Random Number (cont.)

- ◆ To generate a different set of random numbers for every run we add a call to the function **srand()**. **srand()** sets the seed for the random number generator.

- ◆ Syntax:

```
int srand(unsigned seed);
```

- ◆ **Note:**

- **unsigned** is a type that only holds **non-negative** integer values.
- Call **srand()** once with **time(NULL)** as the argument before the first call to **rand()**, typically at the start of your program.

Random Number: Example

- Program outputs a different set of 10 pseudo-random numbers with every run:

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h> // prototype for time(NULL)
```

```
int main()
{
    int i;
```

```
    srand(time(NULL)); // will set a unique seed for each run of the program
```

```
    for (i = 0; i < 10 ; i++) // iterate with "i" as index
```

```
    {
        printf("Random number %d is %d\n", i+1, rand());
    }
```

```
    return 0;
```

```
}
```

```
Random number 1 is 6840
Random number 2 is 24710
Random number 3 is 18358
Random number 4 is 8523
Random number 5 is 31825
Random number 6 is 13407
Random number 7 is 28550
Random number 8 is 6160
Random number 9 is 17571
Random number 10 is 28611
```

Math Library

- ◆ The math library contains many functions that perform mathematical calculations.
- ◆ Their prototypes are listed in **<math.h>**

Common Functions	Syntax	Description	Example
<code>fabs()</code> <code>fabsf()</code>	<code>double fabs(double);</code> <code>float fabsf(float);</code>	return the absolute value of the argument	<code>fabs(-12.5) → 12.5</code> <code>fabsf(-12.5f) → 12.5</code>
...	...	...	...

Math Library- math.h (cont.)

Common Functions	Syntax	Description	Example
floor() floorf()	<code>double floor(double);</code> <code>float floorf(float);</code>	return the largest integer value not greater than the argument	<code>floor(16.3) → 16.0</code>
ceil() ceilf()	<code>double ceil(double);</code> <code>float ceilf(float);</code>	return the smallest integer value not less than the argument	<code>ceil(16.3) → 17.0</code>
round() roundf()	<code>double round(double);</code> <code>float roundf(float);</code>	return the integer value closest to the argument	<code>round(16.3) → 16.0</code> <code>round(-16.3) → -16.0</code>
trunc() truncf()	<code>double trunc(double);</code> <code>float truncf(float);</code>	return the integer part of the argument	<code>trunc(16.7) → 16.0</code> <code>trunc(-16.7) → -16.0</code>
sqrt() sqrtf()	<code>double sqrt(double);</code> <code>float sqrtf(float);</code>	return the square root of the argument	<code>sqrt(16.0) → 4.0</code>

Math Library - math.h (cont.)

Common Functions	Syntax	Description	Example
pow() powf()	<code>double pow(double base, double exponent);</code> <code>float powf(float base, float exponent);</code>	return the result of the first argument raised to the power of the second argument	<code>pow(12.5, 3)</code> → 1953.125
log() logf()	<code>double log(double);</code> <code>float logf(float);</code>	return the natural logarithm of the argument	<code>log(2.718281828459045)</code> → 1.0
exp() expf()	<code>double exp(double);</code> <code>float expf(float);</code>	return the natural anti-logarithm of the argument.	<code>exp(1.0)</code> → 2.718281828459045

Math Library: Demo

Write, compile and run
this program

```
1  /* math_demo.c */
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  int main()
6  {
7      double x= 15.3, y=-2.6;
8
9      printf("floor: %lf, %lf\n", floor(x), floor(y));
10     printf("ceil: %lf, %lf\n", ceil(x), ceil(y));
11     printf("round: %lf, %lf\n", round(x), round(y));
12     printf("trunc: %lf, %lf\n", trunc(x), trunc(y));
13     printf("sqrt: %lf\n", sqrt(x));
14     printf("pow- x^y : %lf\n", pow(x,y));
15     printf("exp- e^x: %lf\n", exp(x));
16     printf("log(x): %lf\n", log(x));
17     printf("log2(x): %lf\n", log(x)/log(2));
18
19     system("pause");
20     return 0;
21 }
```

2 - Time Library: time.h

- ◆ **Date and time information** is presented in computer using an integral number. It is usually the data type long.
- ◆ A **clock tick** is the unit by which processor time is measured and is returned by 'clock'.
- ◆ **CLOCKS_PER_SEC**: Number of clock ticks per second. A constant is defined in time.h (in Dev C++, CLOCKS_PER_SEC =1000 means that 1 clock tick = 1millisecond).
- ◆ 2 data type are defined in the library **time.h**
 - **time_t** : (in DevC++: `typedef long time_t;`)
 - **clock_t** : (in DevC++: `typedef long clock_t;`)

Time Library: time.h (cont.)

Common used functions

time_t time (time_t *tptr); returns the current calendar time and this time is stored in it's parameter.

double difftime (time_t , time_t); returns the difference in seconds between two calendar time arguments.

clock_t clock (void); returns the current date time information using the unit clock tick

We can use these function to evaluate time cost for an algorithm.

Write, compile and run
this program

Time Library: Demo

```
1  /* stdlib_demo.c */
2  /* Evaluate time cost of 1000000000 mathematic operations */
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <time.h>
6  int main()
7  {
8      int i; int n=1; double x=1.5;
9
10     /* Use time_t data type */
11     time_t t1 = time(NULL); /*Get current time */
12     for (i=0; i<1000000000;i++)
13         x= x+1;
14     time_t t2 = time(NULL); /*Get current time */
15     double dt = difftime(t2,t1);
16     printf("\nCost of 1 billion real number adding operations: %lf sec\n", dt);
17
18     t1 = time(NULL); /*Get current time */
19     for (i=0; i<1000000000;i++)
20         n= n+1;
21     t2 = time(NULL); /*Get current time */
22     dt = difftime(t2,t1);
23     printf("\nCost of 1 billion integral number adding operations: %lf sec\n", dt);
24
25     /* Use clock_t data type */
26     n=1;
27     clock_t ct1= clock(); /*Get current time */
28     for (i=0; i<1000000000;i++)
29         n= n+1;
30     clock_t ct2= clock(); /*Get current time */
31     printf("\nCost of 1 billion integral number adding operations: %ld ticks\n", ct2-ct1);
32     printf("or %lf secs\n", ((double)(ct2-ct1)/CLOCKS_PER_SEC));
33
34     system("pause");
35     return 0;
36 }
```

3 - The Character Library (ctype.h)

- ◆ The **<ctype.h>** header provides many functions for classifying and modifying characters.

Common Functions	Description
isalnum()	Checks whether a character is alphanumeric
isalpha()	Checks whether a character is a letter
isblank()	Checks whether a character is a space or tab
isdigit()	Checks whether a character is a decimal digit
islower()	Checks whether a character is a lowercase letter
isupper()	Checks whether a character is an uppercase letter
isspace()	Checks whether a character is a whitespace character
tolower()	Returns a lowercase version of a character
toupper()	Returns an uppercase version of a character

isalnum() Function

- ◆ The **isalnum()** function returns a non-zero value (equivalent to boolean true) if a character is alphanumeric, meaning an **alphabet letter** (a-z) or a **number** (0-9).
- ◆ Example of characters that are not alphanumeric: **(space) !#%&? ...etc.**
- ◆ Syntax: **int isalnum(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char c = 'A';
    if (isalnum(c)) {
        printf("%c is alphanumeric", c);
    } else {
        printf("%c is not alphanumeric", c);
    }

    return 0;
}
```

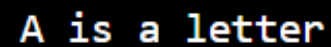
A is alphanumeric

isalpha() Function

- ◆ The **isalpha()** function returns a non-zero value (equivalent to boolean *true*) if a character is **an alphabet letter** (a-z).
- ◆ Syntax: **int isalpha(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char c = 'A';
    if (isalpha(c)) {
        printf("%c is a letter", c);
    } else {
        printf("%c is not a letter", c);
    }
    return 0;
}
```

A screenshot of a terminal window with a black background. The text "A is a letter" is displayed in a light blue or cyan monospaced font.

isblank() Function

- ◆ The **isblank()** function returns a non-zero value (equivalent to boolean true) if a character is a **space (blank)** or a **tab**.
- ◆ Syntax: **int isblank(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char c = ' ';
    if (isblank(c)) {
        printf("%c is blank character", c);
    } else {
        printf("%c is not a blank character", c);
    }
    return 0;
}
```

is a blank character

isdigit() Function

- ◆ The **isdigit()** function returns a non-zero value (equivalent to boolean true) if the character is **a digit**.
- ◆ Syntax: **int isdigit(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char c = '8';
    if (isdigit(c)) {
        printf("%c is a digit", c);
    } else {
        printf("%c is not a digit", c);
    }
    return 0;
}
```

```
8 is a digit
```

islower() Function

- ◆ The **islower()** function returns a non-zero value (equivalent to boolean true) if a character is a **lowercase letter**.
- ◆ Syntax: **int islower(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char c = 'b';
    if (islower(c)) {
        printf("%c is a lowercase letter", c);
    } else {
        printf("%c is not a lowercase letter", c);
    }
    return 0;
}
```

```
b is a lowercase letter
```


isupper() Function

- ◆ The **isupper()** function returns a non-zero value (equivalent to boolean true) if a character is **an uppercase letter**.
- ◆ Syntax: **int isupper(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char c = 'b';
    if (isupper(c)) {
        printf("%c is an uppercase letter", c);
    } else {
        printf("%c is not an uppercase letter", c);
    }
    return 0;
}
```

```
b is not an uppercase letter
```

isspace() Function

- ◆ The **isspace()** function returns a non-zero value (equivalent to boolean true) if a character is a **whitespace** character. Whitespace characters are **spaces**, **tabs** and **newline** characters.
- ◆ Syntax: **int isspace(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char c = ' ';
    if (isspace(c)) {
        printf("%c is whitespace", c);
    } else {
        printf("%c is not whitespace", c);
    }
    return 0;
}
```

is whitespace

tolower() Function

- ◆ The **tolower()** function returns the ASCII value of a lowercase version of the character.
- ◆ Syntax: **int tolower(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char u = 'A';
    char l = tolower(u);
    printf("%c in lowercase is %c", u, l);
    return 0;
}
```

```
A in lowercase is a
```

toupper() Function

- ◆ The **toupper()** function returns the ASCII value of an uppercase version of the character.
- ◆ Syntax: **int toupper(int c);**
- ◆ Example:

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char l = 'a';
    char u = toupper(l);
    printf("%c in uppercase is %c", l, u);
    return 0;
}
```

a in uppercase is A

Exercise 1: Character Classification

- ◆ Write a program that takes a character as input and determines:
 1. Whether it is a digit, uppercase letter, lowercase letter, or none of these.
 2. If it is a letter, convert it to the opposite case (uppercase to lowercase and vice versa).

Summary

The standard C libraries

- ◆ Standard: **stdlib.h**
- ◆ Time: **time.h**
- ◆ Math: **math.h**
- ◆ Character: **ctype.h**

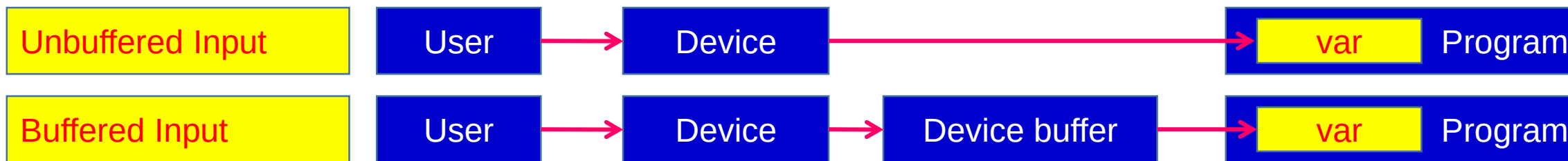
Q&A

Input and Validation

Contents

- ◆ Types of Input
- ◆ Input a character: `getchar()`
- ◆ Input data: `scanf(...)`
- ◆ Input Validation

1 - Types of Input



- ◆ Interactive program (event-based program) uses **unbuffered input**. The program can respond to each and every keystroke directly.
- ◆ **Buffer**: A memory region is associated with a hardware such as keyboard, monitor, hard disk, ... It holds data temporarily.
- ◆ **Buffered input** enables data editing before submission to a program. That means that input data can be treated as units and they can be pre-processed before they are passed to the program.
- ◆ Input functions will access device buffer to get data.

Buffered Input

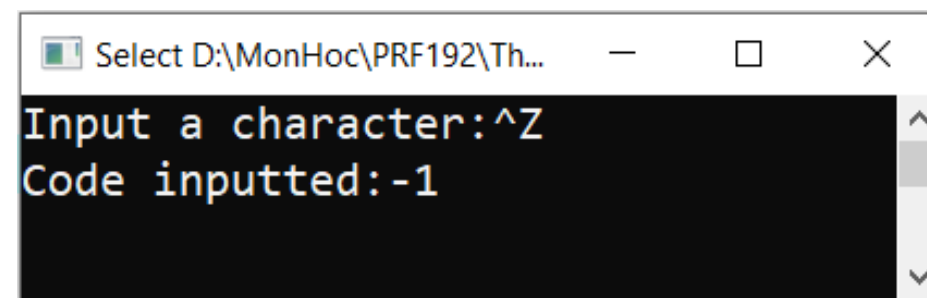
- ◆ To transfer the contents of a buffer to a program the user must press the '\n' character.
- ◆ **Stream**: A concept allows generalizing access data in a buffer as a chain of characters.
- ◆ Two C functions provide buffered input, using standard library **<stdio.h>**
 - **getchar()**: get a character from the keyboard
 - **scanf (...)**: get data from the keyboard

2 - The **getchar()** function

- ◆ **getchar()** retrieves a single character from the standard input stream buffer without translating the input.
- ◆ Syntax: **int getchar(void);**
- ◆ **getchar** returns either the character code for the retrieved character or **EOF**.
(EOF=-1, ctrl+z in Windows, ctrl+d in Unix)

Copy. Paste,
compile and
run the
program
with input:
Ctrl + Z

```
#include <stdio.h>
int main()
{
    char c;
    printf("Input a character:");
    c = getchar();
    printf("Code inputted:%d\n", c);
    return 0;
}
```



```
Select D:\MonHoc\PRF192\Th...
Input a character:^Z
Code inputted:-1
```

getchar(): Clearing the Buffer

- ◆ When the buffer input is used, in some cases, you may not get successfully data for a variable because some characters are remained in the keyboard buffer. **How to remove them?**

```
/* clear empties input buffer */  
void clear (void) {  
    while ( getchar() != '\n' );  
}
```

- ◆ In some tools, the function `fflush(stdin)` is implemented for this purpose (in `stdio.h`).
- ◆ To assure that a program always receives new character data. Make clear the keyboard buffer before the operation of accepting a character (or a string)

getchar() function: Problem

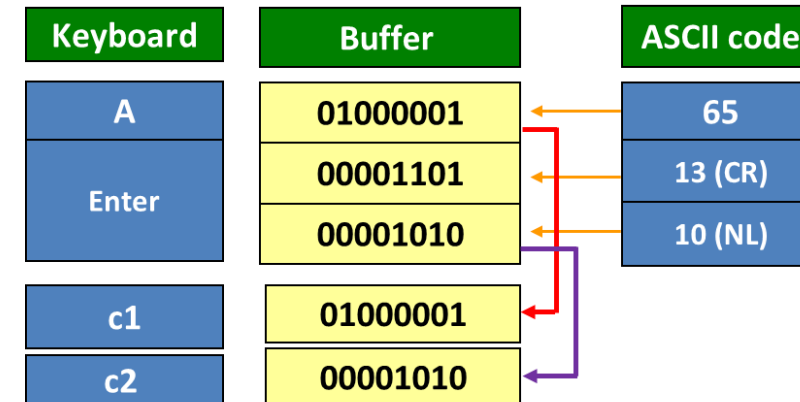
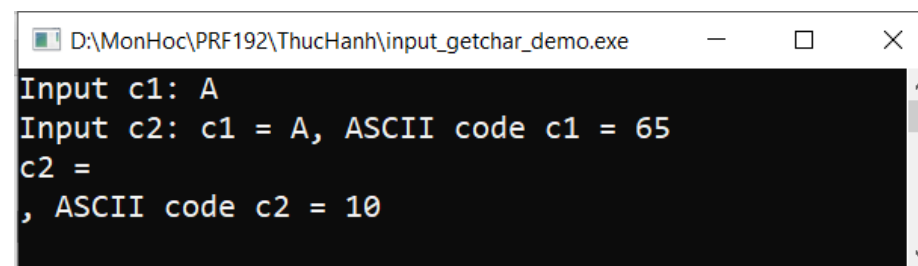
- Write a program input two character variables from STDIN. Print out the value and ASCII code.

```
#include <stdio.h>
int main()
{
    char c1, c2;

    printf("Input c1: ");
    scanf("%c", &c1);
    printf("Input c2: ");
    c2 = getchar();

    printf("c1 = %c, ASCII code c1 = %d\n", c1, c1);
    printf("c2 = %c, ASCII code c2 = %d\n", c2, c2);

    return 0;
}
```



The problem is that after user presses 'A' and ENTER (2 code: 13, 10), ASCII codes of them are put into the keyboard buffer. The function scanf(...) will get 'A' to c1. The remaining codes, 13 and 10, are interpreted to the character '\n' – code 10 only, to c2 by the function getchar(). So, you can not get a new character for c2.

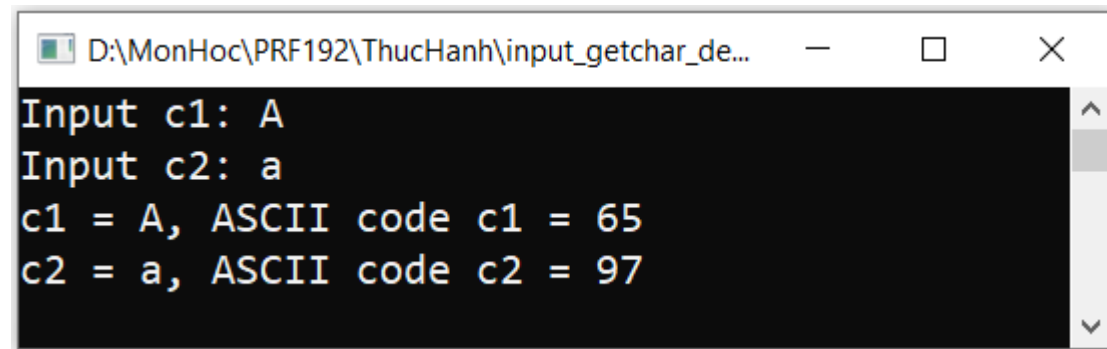
getchar() function: Solution 1

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    char c1, c2;

    printf("Input c1: ");
    fflush(stdin);
    scanf("%c", &c1);
    printf("Input c2: ");
    fflush(stdin);
    c2 = getchar();

    printf("c1 = %c, ASCII code c1 = %d\n", c1, c1);
    printf("c2 = %c, ASCII code c2 = %d\n", c2, c2);

    return 0;
}
```



```
D:\MonHoc\PRF192\ThucHanh\input_getchar_de...
Input c1: A
Input c2: a
c1 = A, ASCII code c1 = 65
c2 = a, ASCII code c2 = 97
```

fflush(stdin) used to clear input buffer

Warning: The **fflush(stdin)** function should not be overused. This function is not defined in the C standard library and is not supported by other compilers.

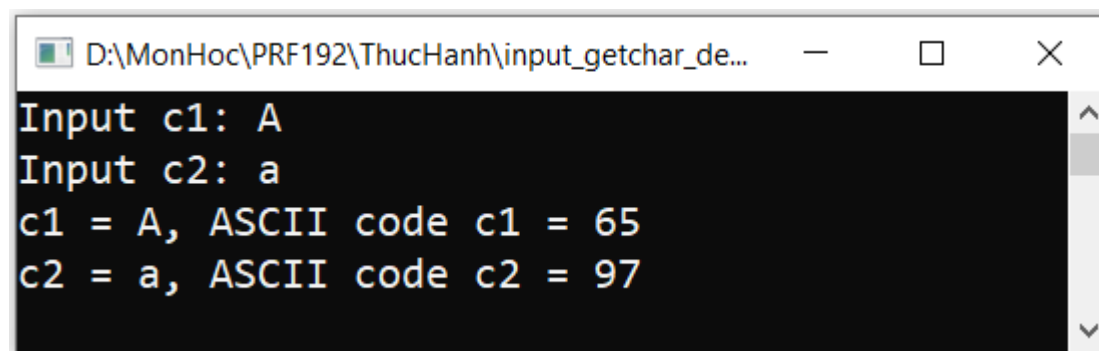
getchar() function: Solution 2

```
#include <stdio.h>
```

```
void clear(){  
    while(getchar()!='\n');  
}
```

The user-defined function for clearing the keyboard buffer

```
int main()  
{  
    char c1, c2;  
  
    printf("Input c1: ");  
    scanf("%c", &c1);  
    clear();  
    printf("Input c2: ");  
    c2 = getchar();  
  
    printf("c1 = %c, ASCII code c1 = %d\n", c1, c1);  
    printf("c2 = %c, ASCII code c2 = %d\n", c2, c2);  
  
    return 0;  
}
```



```
D:\MonHoc\PRF192\ThucHanh\input_getchar_de...  
Input c1: A  
Input c2: a  
c1 = A, ASCII code c1 = 65  
c2 = a, ASCII code c2 = 97
```

Exercise 2: Input Characters

- ◆ Develop a program that will accept a string of characters until the key ENTER is pressed then number of digits, number of alphabets, and number of other characters are printed out.
- ◆ *Hint:*

**Using library functions
in ctype.h**

```
Variable: char c;  
          int numOfDigits=0, numOfAlpha=0, numOfOthers = 0  
Algorithm  
  While (c=getchar() != '\n')  
  { if c is a digit then numOfDigits++ ;  
    else if c is an alphabet then numOfAlpha++;  
    else numOfOthers++;  
  }  
  Print out numOfDigits;  
  Print out numOfAlpha;  
  Print out numOfOthers;
```


3 - **scanf(...)** function

- ◆ **scanf** retrieves data values from the standard input stream buffer under format control.
- ◆ Syntax: **scanf**(**format string**, **&identifier** , ...)
- ◆ **scanf** extracts data values from the standard input stream buffer until **scanf** has.
 - Interpreted and processed the entire format string,
 - Found a character that does not meet the next conversion specification in the format string, in which case **scanf** leaves the offending character in the buffer, or emptied the buffer, in which case **scanf** waits until the user adds more data values.

scanf(...) function (cont.)

- ◆ Some things are concerned when the **scanf(...)** function is used:
 - How to specify the input format string?
 - How to separate input data?
 - How to realize that how many data were inputted successfully?
 - How to remove the byte '\n' in the keyboard buffer after an input operation? Is clearing the buffer is the only way?
- ◆ We will get the answers in the following slides.

scanf(...): Conversion Specifiers

Specifier	Input value	Use with
%c %*c	character	char Remove one character in the input buffer
%d	decimal	char, int, short, long, long ong
%u	decimal	unsigned int, char, int, short, long, long long
%o	octal	unsigned int, char, int, short, long, long long
%x %X	hexadecimal	unsigned int, char, int, short, long, long long
%ld	long	long
%f	float	float
%lf	double	double
%llf	long double	long double

Data from the keyboard are passed to the buffer as characters (ASCII codes). So, conversion specifiers are instructions that specifying how to converse character data to typed-data. If a specifier does not match the data type of a variable, the conversion is incorrect.

scanf(...): Conversion Specifiers (cont.)

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
```

```
    long x;
```

The conversion specifier
does not match the data
type long

```
    printf("Input x: ");
    scanf("%lf", &x);
    printf("x=%ld\n", x);
```

```
    system("pause");
    return 0;
}
```

```
Input x: 5.3
x=858993459
Press any key to continue . . .
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
```

```
    double x;
```

The conversion specifier
does not match the data
type double

```
    printf("Input x: ");
    scanf("%ld", &x);
    printf("x=%lf\n", x);
```

```
    system("pause");
    return 0;
}
```

```
Input x: 5.3
x=0.000000
Press any key to continue . . .
```

scanf(...) function: Default Separators

- ◆ If a input value is number lead by a whitespace , **scanf** treats the whitespace as a separator (The whitespace characters include **newline**, **horizontal tab**, **form feed**, **vertical tab** and **space characters**).

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int x, y;

    printf("Input x and y: ");
    scanf("%d%d", &x, &y);
    printf("x=%d, y=%d\n", x, y);

    system("pause");
    return 0;
}
```

Input x and y: 8 6
x=8, y=6
Press any key to continue . . .

Space
character

Input x and y: 8
6
x=8, y=6
Press any key to continue . . .

New line /
Enter

Input x and y: 8 6
x=8, y=6
Press any key to continue . . .

Tab character

scanf(...) function: User-defined Separators

- ◆ User can specify the character for separating input data.
- ◆ User-define separators will override default separators

```
#include <stdio.h>
#include <stdlib.h>
int n;
int main()
{   int m, k;
    scanf("%d,%d&%d", &n, &m, &k);
    printf("%d, %d, %d\n", n, m, k);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

10,20&30
10, 20, 30
Press any key to continue . . .

```
#include <stdio.h>
#include <stdlib.h>
int n;
int main()
{   int m, k;
    scanf("%d,%d&%d", &n, &m, &k);
    printf("%d, %d, %d\n", n, m, k);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

10,20 ,30
10, 20, 37
Press any key to continue . . .

scanf(...) function: Using %*c for removing a character

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    char c;
    scanf("%d%c", &n, &c);
    printf("%d, %c\n", n, c);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

70D
70, D
Press any key to continue . . .

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    char c;
    scanf("%d%c", &n, &c);
    printf("%d, %c\n", n, c);
    system("pause");
}
```

Input: 70 ENTER

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

70
70,
Press any key to continue . . .

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    char c;
    scanf("%d%*c%c", &n, &c);
    printf("%d, %c\n", n, c);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

70 D
70, D
Press any key to continue . . .

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    char c;
    scanf("%d%*c%c", &n, &c);
    printf("%d, %c\n", n, c);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

70
D
70, D
Press any key to continue . . .

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    char c;
    scanf("%d%*c%c", &n, &c);
    printf("%d, %c\n", n, c);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

70PD
70, D
Press any key to continue . . .

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n;
    char c;
    scanf("%d%*c", &n);
    c = getchar();
    printf("%d, %c\n", n, c);
    system("pause");
}
```

C:\ K:\GiangDay\FU\OOP\BaiTap\demo1.exe

70
D
70, D
Press any key to continue . . .

scanf(...) function: Number of data fields are inputted

◆ Return values from the scanf function:

- **scanf** returns the number of addresses successfully filled or EOF.
 - 0 indicates that scanf did not fill any address,
 - 1 indicates that scanf filled the first address successfully,
 - 2 indicates that scanf filled the first and second addresses successfully,
 - ...
 - EOF (-1) indicates that scanf did not fill any address AND encountered an end of data character.
- ◆ The return code from **scanf** does not reflect success of **%*** conversions.

Number of data fields are inputted (cont.)

```
#include <stdio.h>
#include <stdlib.h>
int main()
{   int m, n; double x;
    int count;
    char c;
    count=scanf("%d%d%lf", &m, &n, &x);
    printf("count=%d, m=%d, n=%d, x=%lf\n", count, m, n, x);
    system("pause");
}
```

C:\K:\GiangDay\FU\OOP\BaiTap\demo1.exe

```
^Z
count=-1, m=2, n=37, x=0.000000
Press any key to continue . . . _
```

C:\K:\GiangDay\FU\OOP\BaiTap\demo1.exe

```
asjklfghjk
count=0, m=2, n=37, x=0.000000
Press any key to continue . . . _
```

C:\K:\GiangDay\FU\OOP\BaiTap\demo1.exe

```
12dfghjkl;
count=1, m=12, n=37, x=0.000000
Press any key to continue . . . _
```

C:\K:\GiangDay\FU\OOP\BaiTap\demo1.exe

```
12 789 asd
count=2, m=12, n=789, x=0.000000
Press any key to continue . . . _
```

C:\K:\GiangDay\FU\OOP\BaiTap\demo1.exe

```
12 789 12.7803
count=3, m=12, n=789, x=12.780300
Press any key to continue . . . _
```

4 - Input Validation

- ◆ We cannot predict how the user will input the data values:
 - Whether the user will enter them as requested or not.
 - One user may make a mistake
 - Or another user may simply try to break the program.
- ◆ The program should have code for trapping all erroneous input, which includes:
 - Invalid characters
 - Trailing characters
 - Out-of-range input
 - Incorrect number of input fields

Input Validation: Example

- ◆ This is a function will ensure that inputted integer must be in the range [min, max] and no invalid characters or trailing character following inputted digits (EXCEPT the ENTER key)

```
1  #include <stdio.h>
2  // Clear input buffer
3  void clear(){
4      while(getchar()!='\n');
5  }
6  // Function prototype
7  int getInt(int min, int max);
8
9  int main(){
10     int n;
11     n = getInt(5, 10);
12     printf("n = %d", n);
13     return 0;
14 }
```



```
D:\MonHoc\PRF192\ThucHanh\input_validation.exe
Input an integer number in range [5->10]: test7
No input accepted!

Input an integer number in range [5->10]: 7.5
Trailing characters!

Input an integer number in range [5->10]: 20
Out of range!

Input an integer number in range [5->10]: 8
n = 8
```

Input Validation: Example (cont.)

```
16 int getInt(int min, int max){
17     int value, flag = 1, count;
18     char lastCharacter;
19
20     do{
21         printf("Input an integer number in range [%d->%d]: ", min, max);
22         count = scanf("%d%c", &value, &lastCharacter);
23
24         if(count==0){
25             printf("No input accepted!\n\n");
26             clear();
27         }else if(lastCharacter!='\n'){
28             printf("Trailing characters!\n\n");
29             clear();
30         }else if(value<min || value>max){
31             printf("Out of range!\n\n");
32         }else{
33             flag = 0;
34         }
35     }while(flag==1);
36
37     return value;
38 }
```

Exercise 3: Input Validation

- ◆ Design and code a function named **getDouble** that receives two double values (a lower limit and an upper limit) and returns user input that lies between the limiting values. Your function rejects any input that includes trailing characters or lies outside the specified limits.

Summary

Input and Validation

- ◆ Types of Input
- ◆ getchar
- ◆ scanf
- ◆ Input validation

Q&A

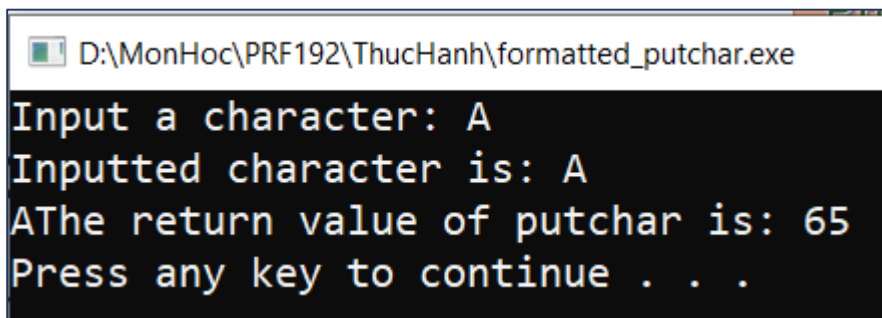
Formatted Output

Formatted Output

- ◆ Standard output is buffered. The standard output buffer empties to the standard output device whenever the buffer receives a newline character or the buffer is full.
- ◆ Buffering enables a program to continue executing without waiting for the output device to finish displaying the most recently received characters.
- ◆ The library **stdio.h** containing functions for print out data:
 - **putchar**(int) : character
 - **printf** (format_string, varList): list of data

putchar(int) function

- ◆ **putchar** writes the character received to the standard output stream buffer and returns the character written or **EOF** if an error occurred.
- ◆ Syntax: `int putchar( int );`
- ◆ For example: `putchar('a');`



```
D:\MonHoc\PRF192\ThucHanh\formatted_putchar.exe
Input a character: A
Inputted character is: A
AThe return value of putchar is: 65
Press any key to continue . . .
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char ch;
    int code;

    printf("Input a character: ");
    ch = getchar();
    printf("Inputted character is: %c\n", ch);
    code = putchar(ch);
    printf("The return value of putchar is: %d\n", code);

    system("pause");
    return 0;
}
```

printf(...) function

- ◆ **printf** sends data under format control to the standard output stream buffer and returns the number of characters sent.
- ◆ Syntax: **printf (format string , value, ..., value)**
- ◆ **Note:**
 - The format string is a literal string that consists of characters interspersed with conversion specifiers.
 - Conversion specifier begins with a % and ends with a **conversion character**

printf(...) function: Format String

- Between the % and the **conversion character**, there may be

% **flags** **width** . **precision** **size** **conversion_character**

flags:

- prescribes left justification of the converted value in its field
- 0** pads the field width with leading zeros

size: identifies the size of data type of the value passed.

Size Specifier	Use With
none	int
hh	char
h	short
l	long
ll	long long

Size Specifier	Use With
none	float
l	double
L	long double

printf(...) function: Conversion Specifiers

Specifier	Output As A	Use With
<code>%c</code>	character	<code>char</code>
<code>%d</code>	decimal	<code>char, int, short, long, long long</code>
<code>%u</code>	decimal	<code>unsigned char, int, short, long, long long</code>
<code>%o</code>	octal	<code>char, int, short, long, long long</code>
<code>%x</code>	hexadecimal	<code>char, int, short, long, long long</code>
<code>%f</code>	floating-point	<code>float, double, long double</code>
<code>%g</code>	general	<code>float, double, long double</code>
<code>%e</code>	exponential	<code>float, double, long double</code>

printf(...) function: Example 1

```
printf("-----\n");
printf("%d|<--      %d\n", 4321);
printf("%10d|<--   %10d\n", 4321);
printf("%010d|<--   %010d\n", 4321);
printf("%-10d|<--   %%-10d\n", 4321);
/* floats */
printf("\n* floats *\n");
printf("00000000011\n");
printf("12345678901\n");
printf("-----\n");
printf("%f|<-- %f\n", 4321.9876546);
/* doubles */
printf("\n* doubles *\n");
printf("00000000011\n");
printf("12345678901\n");
printf("-----\n");
printf("%lf|<-- %lf\n", 4321.9876546);
printf("%10.3lf|<-- %10.3lf\n", 4321.9876546);
printf("%010.3lf|<-- %010.3lf\n", 4321.9876546);
printf("%-10.3lf|<-- %%-10.3lf\n", 4321.9876546);
/* characters */
printf("\n* chars *\n");
printf("00000000011\n");
printf("12345678901\n");
printf("-----\n");
printf("%c|<--      %c\n", 'd');
printf("%d|<--      %d\n", 'd');
printf("%o|<--      %o\n", 'd');
```

```
-----
4321|<--      %d
      4321|<-- %10d
0000004321|<-- %010d
4321      |<-- %%-10d

* floats *
00000000011
12345678901
-----
4321.987655|<-- %f

* doubles *
00000000011
12345678901
-----
4321.987655|<-- %lf
      4321.988|<-- %10.3lf
004321.988|<-- %010.3lf
4321.988  |<-- %%-10.3lf

* chars *
00000000011
12345678901
-----
d|<--      %c
100|<--      %d
144|<--      %o
```

printf(...) function: Example 2

```
#include <stdio.h>

main()
{ int n;
  long lo;
  char ch;
  double d;
  printf("input a integer, a long number, a character: ");
  printf("and a double number: (use blank)");
  scanf("%d %ld %c %lf",&n, &lo, &ch,&d);
  printf("%-20s:%10d%15ld%5c%12.6lf\n","The inputted values are",n,lo,ch,d);
  return 0;
}
```

```
Input a integer number, a long number, a character, and a double number: (use blank) 5 135000 A 45.23
The inputted values are:                5                135000                A                45.230000
```

Size= 20 left align	Size=10 right align	Size=15 right align	5 right align	Size=12 including the dot character, right align
------------------------	------------------------	------------------------	------------------	---

Summary

Formatted Output

- ◆ putchar
- ◆ printf

Q&A

Exercise 4:

Write a C program using the following simple menu:

1- Processing date data

2- Character data

3- Quit

Choose an operation:

- **When user chooses 1:** User will enter values of date, month, year then the program will announce whether this date is valid or not.
- **When user chooses 2:** User will enter two characters, then the program will print out ASCII codes of characters between them using descending order. Example:

Input: ca

Output:

c: 99, 63h

b: 98, 62h

a: 97, 61h

Exercise 5:

Write a C program using the following simple menu:

1- Quadratic equation

2- Bank deposit problem

3- Quit

Choose an operation:

- **When user chooses 1:** User will enter values describing a quadratic equation then the program will print out its solution if it exists.
- **When user chooses 2:** User will enter his/her deposit (a positive number), monthly rate (a positive number but less than or equal to 0.1), number of months (positive integer), then the program will print out his/her amount after this duration.